Goal (single line)

Become a *system designer* who builds reliable, scalable, legal, maintainable dynamic scrapers and data pipelines — able to design, implement, and operate production scrapers and to use AI to extract and validate data.

High-level syllabus (phases)

1. **Prerequisites & thinking style** (3–7 days) — fundamentals you must absolutely own.
2. **Playwright & DOM mastery** (7–10 days) — control the browser reliably.
3. **Extraction & reverse-engineering** (5–8 days) — convert messy pages to clean data.
4. **Engineering for scale, reliability & ethics** (2–3 weeks) — queues, proxies, rate limits, monitoring.
5. **Data pipeline & storage** (1–2 weeks) — DB design, incremental crawls, schema evolution.
6. **Production deployment & ops** (1–2 weeks) — containerize, CI/CD, metrics, health checks.
7. **AI-augmented scraping & QA** (ongoing) — LLMs for extraction, validation, error correction.
8. **Capstone projects** — 3 production-grade projects you can show on GitHub/LinkedIn.

(You can compress/parallelize phases depending on time — but don't skip the engineering & ethics sections.)

Phase 0 — Must-know foundations (Feynman)

Simple: scraping is "ask a website for a page and turn its HTML into structured data."
Deeper: modern sites often render using JavaScript; you either run a browser (Playwright) or reconstruct the network calls the page makes. Know HTTP, DOM, CSS selectors, basic JS, and how browsers load resources (XHR/fetch, DOMContentLoaded, network_idle).
Analogy: HTML is raw ingredients; your parser is the recipe that turns them into a dish. If the kitchen cooks ingredients after you arrive (JS), you must wait or run the same kitchen (browser). **Practical exercises:**

- Built-in check: `curl https://example.com` → inspect HTML.
- Open DevTools (Network tab) on any site and watch what happens when you click a button. **Teach-back questions:** What happens between typing a URL and seeing a rendered page? How would you get JSON if the site loads data via XHR?

Phase 1 — Playwright & browser control (Feynman)

Simple: Playwright controls a real browser from Python so you can interact with sites like a human. **Deeper details (key skills):**

- Launching headless/headful browsers, creating pages and contexts.
- Selectors: CSS selectors, text selectors, XPath, nth-child, role selectors.
- Wait strategies: `wait_for_selector`, `wait_for_load_state('networkidle')`, explicit vs implicit waits.
- Interactions: click, fill, evaluate JS in page, screenshot, cookies, localStorage.
- Network inspection: intercept requests/responses, emulate network conditions, modify headers.
- Stealth & fingerprints: user-agent, viewport, timezone, locale, WebGL — tradeoffs and limits. **Analogy:** Playwright is a remote-controlled robot that can open a browser, click, and copy results — you script the robot to behave reliably even when the page is slow or flaky. **Code — minimal Playwright example (Python):**

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch(headless=True)
    page = browser.new_page()
    page.goto("https://news.ycombinator.com",
wait_until="networkidle")
    titles = [e.inner_text() for e in
page.query_selector_all(".storylink")]
    print(titles[:5])
    browser.close()
```

Practical exercises:

1. Use Playwright to load a JS-heavy site (e.g., Twitter or a SPA) and take a screenshot after `networkidle`.
2. Write a script that clicks “Load more” until it can’t, saves page HTML after each click.

3. Intercept XHR requests and log JSON responses (use `route`/`on("response")`). **Teach-back checks:** Explain reasons for `wait_for_selector` vs `sleep`. What breaks if you use `sleep` only?

Phase 2 — Extraction & reverse engineering (Feynman)

Simple: extraction = find the data on a page and pull it out reliably. **Key topics:**

- Parsers: BeautifulSoup, lxml, parsing HTML fragments vs full page.
- Selectors & XPath mastery.
- JSON-LD & structured data (microdata/schema.org): often the easiest path to clean data.
- Reverse-engineering API endpoints: use DevTools to capture API calls, replay them (faster and cleaner than rendering).
- Anti-patterns: brittle selectors tied to UI classes — prefer semantic selectors or fallback extraction. **Analogy:** If HTML is a messy office, your extractor is the filing system that finds documents reliably even when the office rearranges desks. **Exercises:**
- Find data on a product page in three different ways: CSS selector, XPath, and by inspecting XHR JSON. Compare reliability.
- Implement fallback: primary JSON-LD path → secondary CSS selector → tertiary heuristic using LLM (see Phase 6). **Teach-back checks:** How would you extract price when the site uses client-side price formatting? How to detect and handle lazy-loaded content?

Phase 3 — Scale, reliability & ethics (Feynman)

Simple: moving from a script to a system requires handling scale, errors, politeness, and legal boundaries. **Core building blocks:**

- **Scheduler:** When and what to crawl (cron, Airflow, APScheduler).
- **Workers & Concurrency:** concurrency model (threads vs processes vs async), Playwright server vs per-task browser instances, browser contexts.
- **Queue:** Redis/RabbitMQ/Kafka for work distribution.
- **Rate limiting & politeness:** per-domain rate limits, exponential backoff, randomized delays.
- **Proxies & IP rotation:** residential vs datacenter proxies, TTL, cost tradeoffs.

- **CAPTCHAs & paywalls:** *I will not help bypass paywalls/CAPTCHAs*. Instead: contact provider, use official API, or obtain data via partnerships.
- **Robots.txt & ToS compliance:** read robots.txt (it's not law everywhere) and respect legal/ethical constraints. When in doubt, prefer legit APIs.
- **Observability:** structured logs, metrics (success rate, error types), tracing (request IDs), and alerting. **Analogy:** Building at scale is turning a single delivery bike into a logistics fleet — you need routing, dispatch, retries, and monitoring. **Practical tasks:**
- Implement a worker that pulls URLs from Redis, runs Playwright extraction, and stores results. Include metrics (count success/fail).
- Add exponential backoff and circuit breaker for a failing domain. **Teach-back checks:** Why use a queue? What happens when Playwright is run with 100 concurrent browser instances?

Phase 4 — Data pipeline & storage (Feynman)

Simple: scrape → normalize → validate → store. **Details:**

- Choose storage by access pattern: OLTP (Postgres), document store (Mongo), blob store (S3) + catalog (Parquet/Delta) for analytics.
- Schema & versioning: keep a schema registry; store raw HTML + parsed data to allow re-parsing when logic changes.
- Idempotency & dedup: generate stable record keys, dedupe on insert.
- Incremental crawls & change detection: ETags, Last-Modified, hashing content, change score.
- Data validation: type checks, ranges, canonicalization, and anomaly detection. **Exercises:**
- Design a table schema for product listings supporting historical price tracking. Implement insert/upsert logic.
- Implement a pipeline that saves raw HTML to S3 and parsed records to Postgres; run 100 items and show recovery after parser change. **Teach-back checks:** How will you reprocess old HTML when parsing rules change? Why store raw HTML?

Phase 5 — Production & deployment (Feynman)

Simple: containerize, orchestrate, monitor, automate. **Essentials:**

- Dockerize Playwright (use official Playwright images). Watch file descriptors and ephemeral storage.
- Kubernetes jobs or a managed worker autoscaling pattern. Consider using smaller browser contexts instead of full browser instances to save RAM.
- CI/CD: linting, tests, image build, canary deploys.
- Alerting & runbooks: SLOs (success rate), error budgets, documented remediation steps. **Exercises:**
- Build a Dockerfile for Playwright Python script and run it locally.
- Create a simple health-check endpoint for your worker and wire a Prometheus metric (success/fail counts). **Teach-back checks:** When would you prefer serverless vs k8s for your scraper fleet?

Phase 6 — AI augmentation (how to leverage current AI)

Simple: use LLMs to extract fuzzy fields, validate inconsistent data, map variants to canonical entities, and generate heuristics. **Use cases:**

- **Extraction assistant:** feed HTML snippets to an LLM to extract "address", "ingredients", or "symptoms" when selectors fail. Set strict output schema (JSON schema) and validate.
- **Schema mapping:** map scraped variants to canonical categories (e.g., size "M", "Medium", "med" → "M").
- **Change detection:** LLMs can summarize diffs and flag format changes that break rules.
- **Automation & governance:** use LLM to suggest parser updates but require human review before production rollouts. **Warnings:** LLMs hallucinate. Always validate LLM output using deterministic checks (regex, enums, checksum). Log LLM confidence and provenance (what snippet was fed). **Practical exercise:**
- Build a microservice that inputs HTML and outputs a JSON via an LLM with strict schema validation. Compare accuracy vs pure selector-based extractor. **Teach-back checks:** When should you *not* use an LLM in your pipeline? How do you mitigate hallucinations?

Production-grade project roadmap (concrete projects & deliverables)

1. **Project A — Playwright Starter (single-domain)**

- Deliverable: Playwright script that scrapes 500 product pages, saves raw HTML + parsed JSON to disk, with retry/backoff.
- Must: logging, per-domain rate limit, README + run instructions.

2. **Project B — Worker + Queue (multi-domain)**
    - Deliverable: Redis queue, worker pool with Playwright contexts, Postgres storage, basic dashboard (success/fail).
    - Must: idempotency, doc storage, basic metrics.
3. **Project C — Production pipeline + AI assist**
    - Deliverable: Containerized pipeline (Docker), k8s manifest (or CLI scripts), LLM-assisted fallback extractor, tests, and a public GitHub repo + one-page project write-up.
    - Must: runbook, SLOs, cost estimate for 1k pages/day.

For each project you should provide:

- Acceptance tests (sample URLs + expected JSON output).
- CI that runs unit tests on extraction logic with saved HTML fixtures.
- A short blog-style README explaining architecture and tradeoffs.

First 10 actionable tasks (do these now)

1. Install Playwright and run the sample script above. Confirm it prints titles.
2. Inspect DevTools on a JS-heavy site and capture XHR endpoints. Save 3 JSON responses.
3. Write a Playwright script that clicks “Load more” until no more button. Save HTML snapshots.
4. Parse one snapshot with BeautifulSoup; extract 5 fields reliably.
5. Store one parsed record in Postgres (local). Create an `INSERT ... ON CONFLICT` upsert.
6. Wrap the scraper as a function and add basic logging + retry decorator.
7. Push code to GitHub with a clear README and one screenshot.
8. Add a small test that runs parser on saved HTML fixture and asserts JSON keys.
9. Read robots.txt of target domain and summarize constraints in README.
10. Draft a 1-page architecture diagram (markdown + ascii) showing scheduler → queue → workers → storage.

How I expect you to think like a system designer (short checklist)

- Always separate **control plane** (scheduler, orchestrator) from **data plane** (workers, extraction).
- Make parsing *replayable*: store raw inputs; parsing logic should be decoupled and testable.
- Make failures visible: successes, errors, and why parsers fail.
- Assume change: every parser will fail when the site changes — design for quick rollback + safe deploys.
- Keep cost in mind: browser instances cost RAM; simulate at low-cost first (replay API calls if possible).

Quick rubric to evaluate your progress

- **0 → 3 days:** You can run Playwright and capture page HTML.
- **1 week:** Reliable single-domain extractor + tests.
- **3 weeks:** Scalable worker + queue + storage + basic metrics.
- **2 months:** Production deployment with CI, can handle 1k pages/day, and has LLM fallback with validation.
